

Session_09 activities worksheet

Online handout: plots of damped oscillations; online listings: filename_test.cpp, diffeq_pendulum.cpp, GnuplotPipe class

A) Strings and Things

The filename_test.cpp code has examples of the use and manipulation of C++ strings, including building filenames the way we do stream output. **Be careful NOT to put << endl when creating filenames.**

1. Using make_filename_test, compile and link filename_test.cpp and run it. Look at the output files and the printout of the code to see how it works.
2. Modify the code so that there is a loop running from 0 to 3 with index variable j. For each j, open a file with a name that includes the current value of j. Write "This is file j", where "j" here is the current value, into each file and then close it. Did you succeed?

→ Yes, it works.

3. Modify the code to input a double named alpha and open a filename with 3 digits of alpha as part of the name. (E.g., something like pendulum_alpha5.22_plot.dat if alpha = 5.21934.) Output something appropriate to the file. Did it work?

→ Yes, it works.

B) Upgrades from the diffeq_oscillation to diffeq_pendulum code

- There are three new menu items: plot_start, plot_end, and Gnuplot_delay. The equation is still solved from t_start to t_end, but results are only printed out from plot_start to plot_end. Initially these are the same time intervals, *but you can use plot_start to exclude a transient region*. So if the system settles down to periodic behavior at t=20, setting plot_start=20 means that $0 < t < 20$ is not plotted, which makes the phase-space plots much easier to interpret.
 - We've also incorporated code to do real-time plotting in gnuplot directly from C++ programs. We have made a class to do this but it is rather crude: the interface and documentation needs work, and it probably has bugs! *Look at the GnuplotPipe.h printout and the GnuplotPipe.cpp file to get an idea how it works.* Gnuplot_delay sets the time in milliseconds between plotted points.
-

C) Damped (Undriven) Pendulum

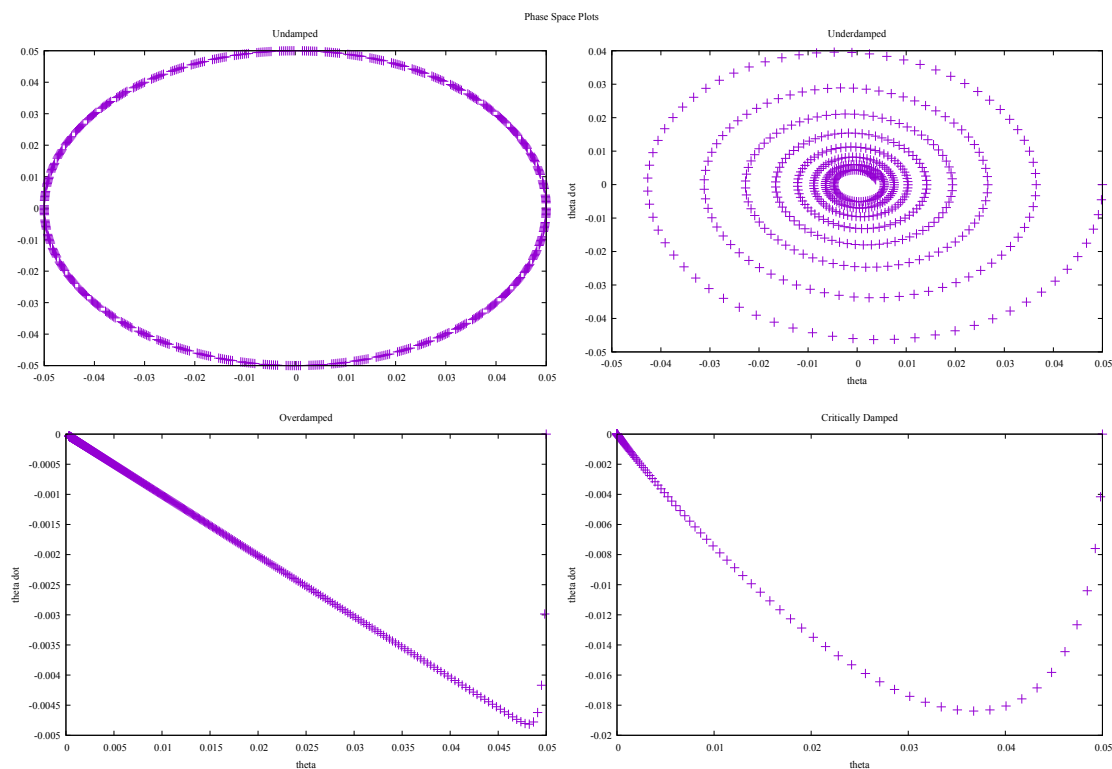
The pendulum modeled here has the analog of the viscous damping: $F_f = -b \cdot v$, where $v(t)$ is the velocity, that was used in Session_07. The damping parameter is called alpha here.

1. Use `make_diffeq_pendulum` to compile and link `diffeq_pendulum.cpp`. Run it while taking a look at the printout of the code. It should look a lot like `diffeq_oscillations.cpp`, with different parameter names. Run it with the default parameters, noting the real-time phase-space plot. There is also an output file `diffeq_pendulum.dat`.
2. *Modify the code so that the output file includes two digits of the variable alpha in the name. Did you succeed?*

→ Yes, it works.

Generate the analogs of the four phase-space plots on the handout but with pendulum variables and initial conditions $\theta_{\dot{0}}=0$ (at rest) and θ_0 such that you are in the simple harmonic oscillator regime (note that θ is in radians). Set $f_{\text{ext}}=0$ (no external driving force) and then do four runs with four values of alpha corresponding to undamped, underdamped, critically damped, and overdamped (convert from the conditions on b discussed in the background notes). *What values of θ_0 and alpha did you use?*

→ We used a θ_0 of 0.05 so the small angle approximation holds (best for harmonic oscillator regime). We used alpha values of 0 for undamped, 0.1 for underdamped, 10 for overdamped. Critical damping relates to $\omega_0=1$, so we set it to 2. Below is the plot:



D) Damped, Driven Pendulum

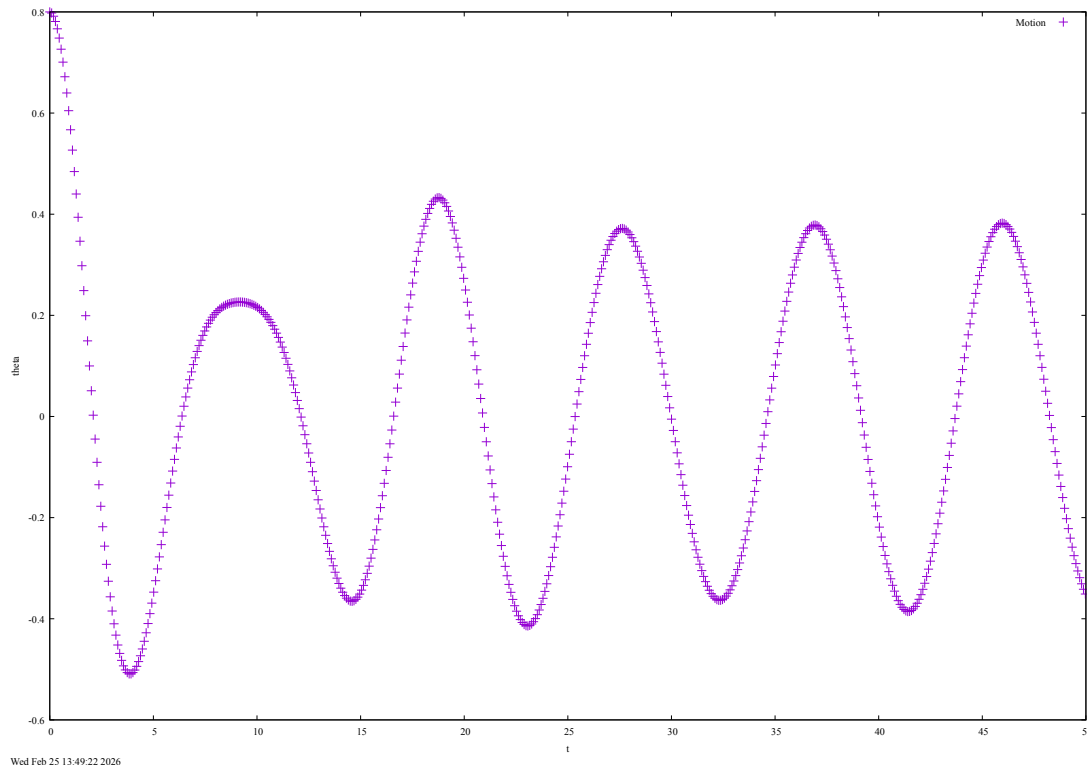
This is a quick exercise to look at transients.

1. Restart the program so that we use the defaults. There is both damping and an external driving force, with frequency $w_{\text{ext}} = 0.689$. The initial plot is from $t=0$ to $t=100$. Run it. *The green points are plotted once every period of the external force. What good are they?*

→ The green points are the so-called Poincaré plots. They help us distinguish steady state solutions from transients and help us identify periodic motion (or multiple periodicities, like period 2 solutions). They are points at the periodicity of the external driving force. It also shows us the underlying structure of the attractors in the phase space.

2. Note that it seems to settle down to a periodic orbit after a while. *Plot ("by hand" with gnuplot) θ vs. t from the output file `diffeq_pendulum.dat` and see how long it takes to become periodic.*

→ Below is the motion plot. It seems that the transient ends around $t=25$ s, and after that the motion becomes periodic. This may be a multiple-period motion (it's hard to know for sure at this scale).



3. Run the code again with "plot_start" set to the time you just found. *Have you gotten rid of the transients? What is the frequency of the asymptotic $\theta(t)$?*

→ It seems most of the transients have gone away after 25s. It seems the period is about 10s, so the frequency of the asymptotic $\theta(t)$ is about $2\pi/10$ s which is about 0.63Hz. This makes sense given our ω_0 of 0.689Hz.

E) Looking for Chaos

Now we want to explore more of the parameter space and look at different structures. In Section f of the Session_07 notes there is a list of characteristic structures that can be found in phase space, with sample pictures in Figure 1.

1. In phase space, a fixed point is a (zero-dimensional) point that "attracts" the time-development of a system. By this we mean that many (or all) initial conditions end up at the same point in phase space. The clearest case is a damped, undriven system like a pendulum, which ends up at $\theta=0$ and zero angular velocity no matter how it starts. If the steady-state trajectory in phase space is a closed (one-dimensional) curve, then we call it a limit cycle.
2. Try some prescribed values for the pendulum. You will need to adjust "plot_start" and extend the plot time (increase "t_end" and "plot_end"). *Fill in the missing descriptions (i.e., the first column in lines 3, 4, and 5) in this table:*

description	alpha	f_ext	w_ext	theta0	theta_dot0
period-1 limit cycle	0.0	0.0	0.689	0.8	0.0
period-3 limit cycle	0.2	0.52	0.689	-0.8	0.1234
chaotic pendulum	0.2	0.52	0.694	0.8	0.8
period-3 limit cycle	0.2	0.52	0.689	0.8	0.8
chaotic pendulum	0.2	0.9	0.54	-0.8	0.1234

Can you tell how many "periods" the limit cycles have from the graphs? How might you identify whether a function of time $f(t)$ is built from one, two, three, ... frequencies?

→ You can tell the number of periods from the green Poincaré points. If the points all overlap in three locations, then that's period 3. If the points all overlap at one point, then it's period one. If it never overlaps, it chaotic. That's how you identify how many frequencies are building $f(t)$.

3. One characteristic of chaos is an "exponential sensitivity to initial conditions." *For the last combination, vary the initial conditions very slightly (e.g., change x_0 by 0.01 or 0.001); what*

happens?

→ When you vary the initial conditions, the outputted phase space plots look very different. This is the sensitivity to initial conditions, where a miniscule “kick” in the initial x_0 or v_0 causes a wildly different result.

F) Armadillo linear algebra library

Here we'll use the Armadillo library as an example of how to install a personal copy of a library and set up a makefile appropriately to use it. We'll try some basic examples and see how to use it as an alternative to GSL in the Hamiltonian class.

1. **Installing Armadillo.** Follow these instructions but ask if something goes wrong.
 1. Create a subdirectory called `my_armadillo` in your 5810 directory.
 2. Download from <http://arma.sourceforge.net/download.html>
2. [Links to an external site.](#) the latest Armadillo tarball (named something like `armadillo-###.###.###.tar.xz`) into `my_armadillo` and unpack it with `tar xfv armadillo-###.###.###.tar.xz` (substitute for the #'s appropriately).
3. Go into the resulting directory (named `armadillo-###.###.###`) and look at `README.md`. Give the command `pwd` to find the full path to this directory; we'll need to use it several times so I'll call it `<path-to-Armadillo>`. When I tested it, my `<path-to-Armadillo>` was:
`/home/bundschuh.2/phy5810/my_armadillo/armadillo-12.8.0`
However, since we never want to use absolute paths to our home directory, you should drop that part of the path and replace it by `${HOME}`. In my case that means `<path-to-Armadillo>` is `${HOME}/phy5810/my_armadillo/armadillo-12.8.0`
4. Now type (with returns after each command!) `cmake . -DCMAKE_INSTALL_PREFIX:PATH=<path-to-Armadillo>` (note the period and replace `<path-to-Armadillo>` by whatever you actually found above), then `make install`. This first creates a makefile with `cmake` and the second one installs it in the same directory (we could put it somewhere else). *Did this seem to work?*

→ Yes, it seems to have worked.

- **Testing Armadillo.** Go to the `Session_09` directory and edit section 3 of `make_armadillo_tests` to prepare it for your installation.
 1. We need to link to three libraries. After `LIBS=` add `-larmadillo` # **these are lower case L; on some machines (not OSC though) you also need to add -llapack -lblas**
 2. We need to set the path to the Armadillo library. After `LDFLAGS=` add `-L<path-to-Armadillo>`
 3. We need to set the path to the Armadillo headers. After `CFLAGS= -g -O2` add `-I<path-to-Armadillo>/include` # **this is an upper case i**

4. The Armadillo library needs some constructs from the C++-11 standard, so we also need to add `-std=c++11` at the end of the CFLAGS line
5. Unfortunately, the Armadillo library uses some constructs in its header files that are less than perfectly clean and we thus have to remove `-Wshadow` from the CWARNS line.
6. Now try to compile and link `armadillo_tests`. It should work to create the `.x` file, but may (or may not) fail when you try to run `armadillo_tests.x`. If it fails, the problem is that you need to set the path to the Armadillo library. At the terminal prompt, type:


```
setenv LD_LIBRARY_PATH ${LD_LIBRARY_PATH}:<path-to-Armadillo>
```

 if you are using `tcsh`. If you are using `bash` (type `echo $SHELL` to check), type:


```
export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:<path-to-Armadillo>
```

 Now it should work!
7. Look at the code and play a bit (e.g., modify the examples). Then add code to solve:

$$\begin{aligned} x_1 + 2x_2 + 3x_3 &= 2 \\ 2x_1 - 3x_2 + 4x_3 &= 5 \\ x_1 + 4x_2 - 2x_3 &= -2 \end{aligned}$$
 as converted to the matrix problem $\mathbf{Ax} = \mathbf{b}$. *What is the result for the solution vector $\mathbf{x}$?*

→ The solution seems to be $x_1=2/3$, $x_2=-1/3$, and $x_3=2/3$.

You can browse the [online documentation](#)

- [Links to an external site.](#) for other examples of using the Armadillo library.
- **Armadillo for the Hamiltonian class.** An alternative version of `eigen_tridiagonal_class.cpp`, called `eigen_tridiagonal_class_armadillo.cpp`, uses Armadillo as an alternative to GSL.

1. The `diff file1 file2` command can be used to show all of the line-by-line differences between two files `file1` and `file2`. [See <http://www.computerhope.com/unix/udiff.htm>

3. [Links to an external site.](#) for more details on `diff`. Type `man diff` to see all of the (many) options.] *Use diff to compare the two eigen_tridiagonal cpp files (both are in Session09.zip). Why are there so few differences? What has been hidden?*

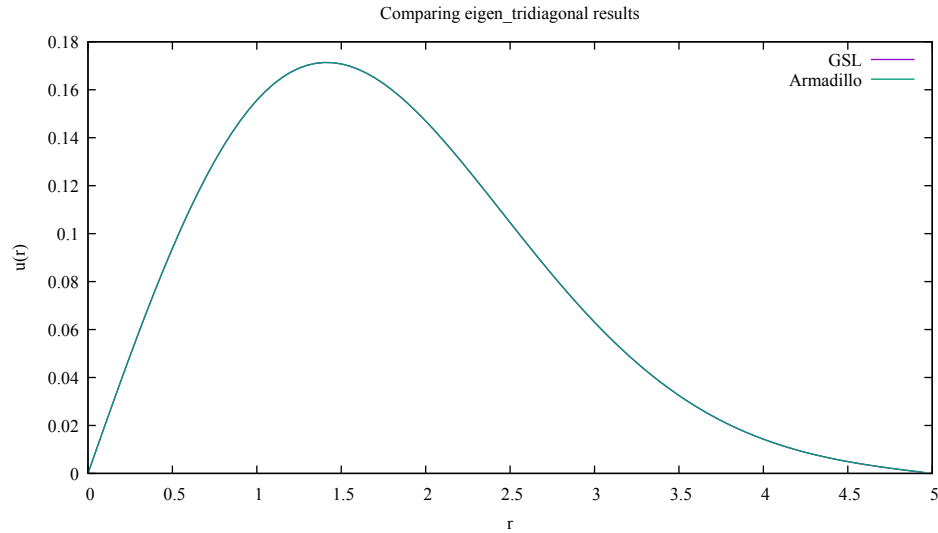
→ There are so few differences because the main differences are all held in the Hamiltonian class include. The majority of the differences of behavior come from differences in `GslHamiltonian.h` vs. `ArmadilloHamiltonian.h`. Because of this, the `eigen_tridiagonal` doesn't have too many differences.

4. Look at the `ArmadilloHamiltonian` header and `cpp` files, looking up the functions in the Armadillo documentation (see the comments). *Which elements of the class do you not understand?*

→ I think I understand the class from a high level (i.e, `get_eigenvalue` gets the eigenvalue). I think I understand the syntax too for storing values in the matrix or making a matrix filled with the value 0. I don't really understand `eig_sym` in terms of how it actually works though, but it seems to get eigenvalues of a symmetric matrix.

5. Edit `make_eigen_tridiagonal_class_armadillo` to use your personal Armadillo library and compile, link, and run `eigen_tridiagonal_class_armadillo`. Compile, link, and run the GSL version to create another .dat file and use `eigen_tridiagonal_comparison.plt` to compare the outputs. *Are they the same? Is this great, or what?*

→ Yes, the outputs are same (This is great)! The plot from the .plt file is shown below and we see practically total overlap.



G) Reflection

Write down which of the six learning goals on the syllabus this worksheet addressed for you and how:

→ This session helped me learn more tools of computational physics, like Armadillo. I now also better understand physics concepts like chaos by putting them through code and I better understand chaos in general through playing with the Gnuplot numerical solutions to different oscillating systems.